Proyecto final de Deep Learning, UVG 2026

¿Esta compra es fraude?

Llevamos a producción una LSTM que, en milisegundos, decide si un pago con tarjeta se aprueba o se bloquea.

Iris Ayala, Anggie Quezada y Erick Guerra

fraude-tarjeta.onrender.com

Un error cuesta Q4,200. El otro, Q180.

Q4,200

si dejamos pasar un fraude

Q180

si bloqueamos a un cliente que no hizo nada malo

Un fraude cuesta **23 veces más** que un bloqueo, así que preferimos pasarnos de cuidadosos.
Y como solo el 0.58 % de las compras son fraude, revisarlas todas a mano no es opción.

Quién lo usa y para qué

El área de riesgos del banco, a través de su sistema de autorización. En cada compra nos manda las últimas transacciones de la tarjeta y le devolvemos un puntaje de riesgo con la decisión.

Para quien no programa hay una página web: eliges un caso y ves qué decide el modelo.

Si el puntaje es 0.025 o más



Bloquear

Si es menor



Aprobar

El puntaje va de 0 a 1 y sale en unos milisegundos.

Lo que estamos suponiendo

Los datos se parecen a la realidad

Sparkov es simulado. Si el fraude real es distinto, hay que reentrenar.

El fraude no cambia mucho

Si aparecen patrones nuevos, detecta menos. Para eso está el monitoreo.

Nos mandan bien los datos

Una tarjeta, hora local y una de las 14 categorías. Si no, error 422.

Los costos son fijos

Si cambian los Q4,200 o los Q180, cambia el umbral ideal.

Los montos son quetzales

Sparkov viene en dólares. Con montos locales habría que reentrenar.

Alcanza con CPU y un plan gratis

Se duerme y borra el registro al reiniciar. Un banco necesitaría servidor fijo.

Una LSTM pequeña que lee la historia de la tarjeta

Las últimas 10 compras

↓

Categoría, monto, hora y días entre compras

↓

LSTM de 32 unidades

↓

Puntaje de riesgo entre 0 y 1

5,469 parámetros, corre en CPU

Lo que corregimos del Proyecto 1

1 **El umbral se elige en validación**
Antes se eligió mirando la prueba, y el costo salía optimista.

2 **El relleno tiene su propio código**
Se confundía con compras de entretenimiento.

3 **Las variables están escaladas**
El monto en miles dominaba a la hora, que va de 0 a 23.

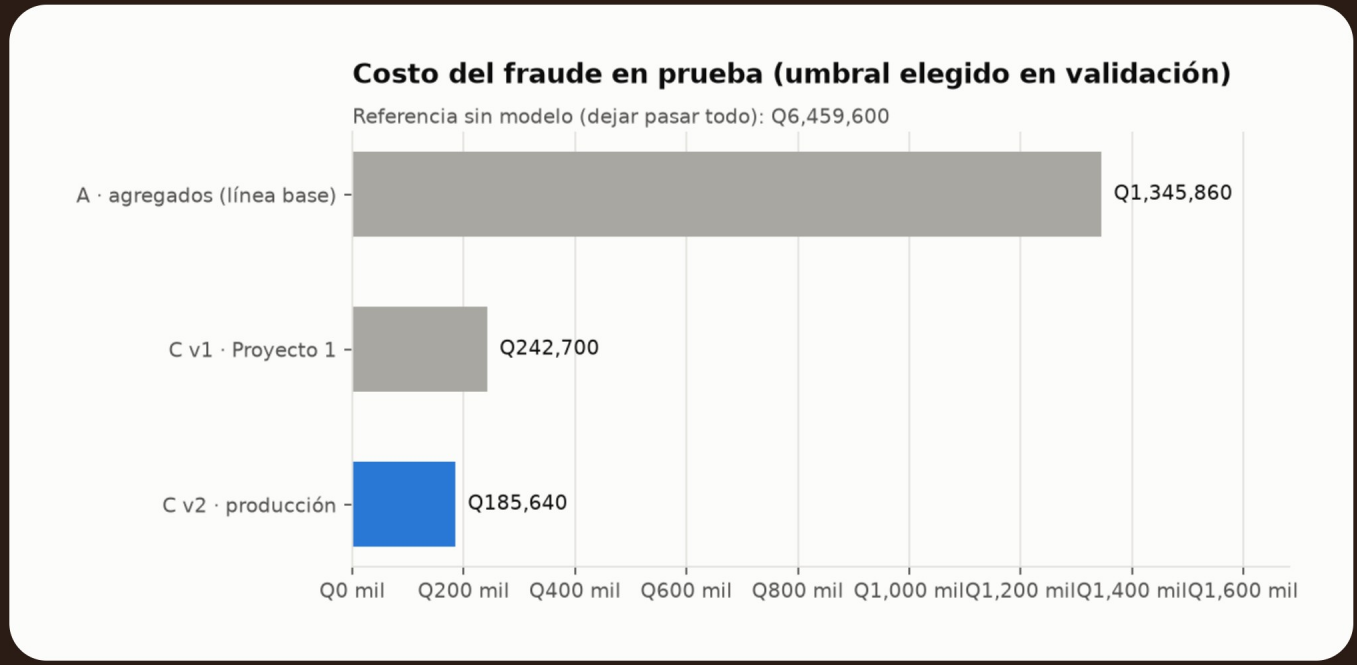
4 **Se usa todo el historial**
Como en la vida real, donde la tarjeta ya tiene compras previas.

Cuesta 86 % menos que la línea base

0.989
de AUC-PR (el Proyecto 1 tenía 0.983)

25 de 1,538
fraudes se escaparon

Q325 mil
menos al mes que la línea base



6 · En 107 días de prueba que el modelo nunca vio. Sin modelo, el costo sería Q6,459,600.

Lo que prometimos antes de entrenar

Escribimos los criterios en docs/hipotesis.md antes de ver un solo resultado.

H1

Costar menos que la línea base

Meta: menos de la mitad del costo

13.8 %

del costo de la línea base, y +0.286 de AUC-PR

Se cumple

H2

No empeorar al corregir

Meta: perder como mucho 0.01 de AUC-PR

+0.006

de AUC-PR frente al Proyecto 1: mejoró

Se cumple

H3

Responder a tiempo

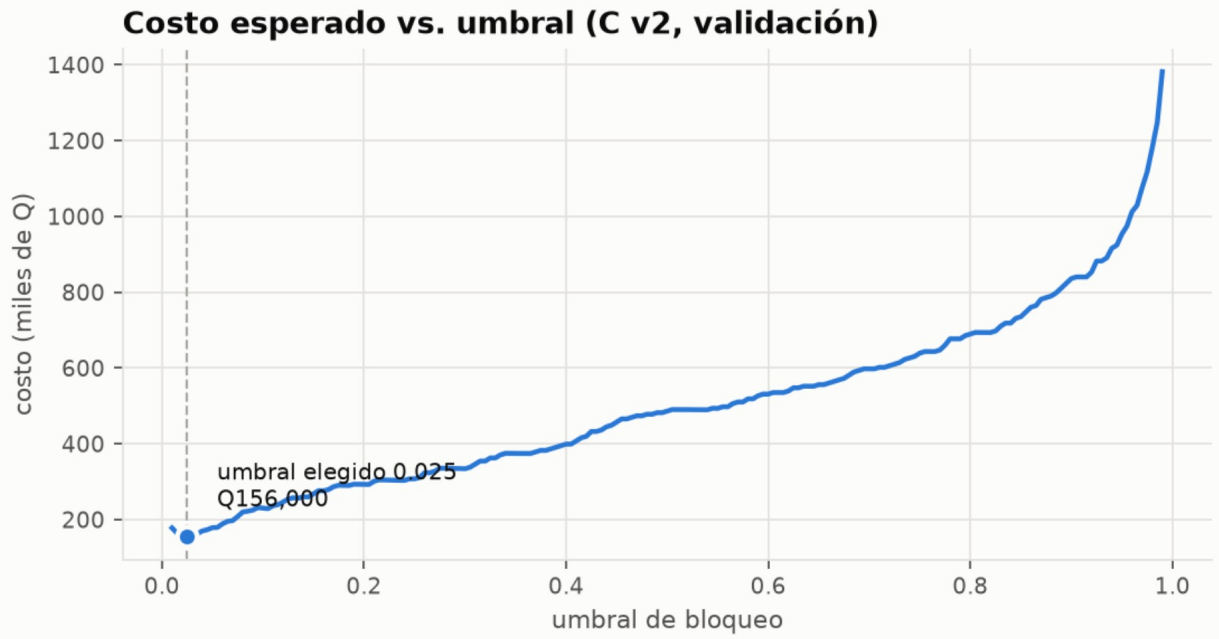
Meta: p95 menor a 300 ms

1.29 ms

de p95 en el servicio publicado

Se cumple

¿Por qué bloquear desde un 2.5 %?



Con el umbral de 0.025

Q185,640

se escapan 25 fraudes

⚠ Con 0.5, el valor por defecto

Q449,160

se escapan 106 fraudes: 2.4 veces más caro

Cómo está armado

Datos

Sparkov: 1.3 millones de compras, separadas por fecha.

Entrenamiento

Un script en CPU genera el modelo y evalúa las hipótesis.

Servicio

FastAPI en Docker, publicado en Render, con la página web.

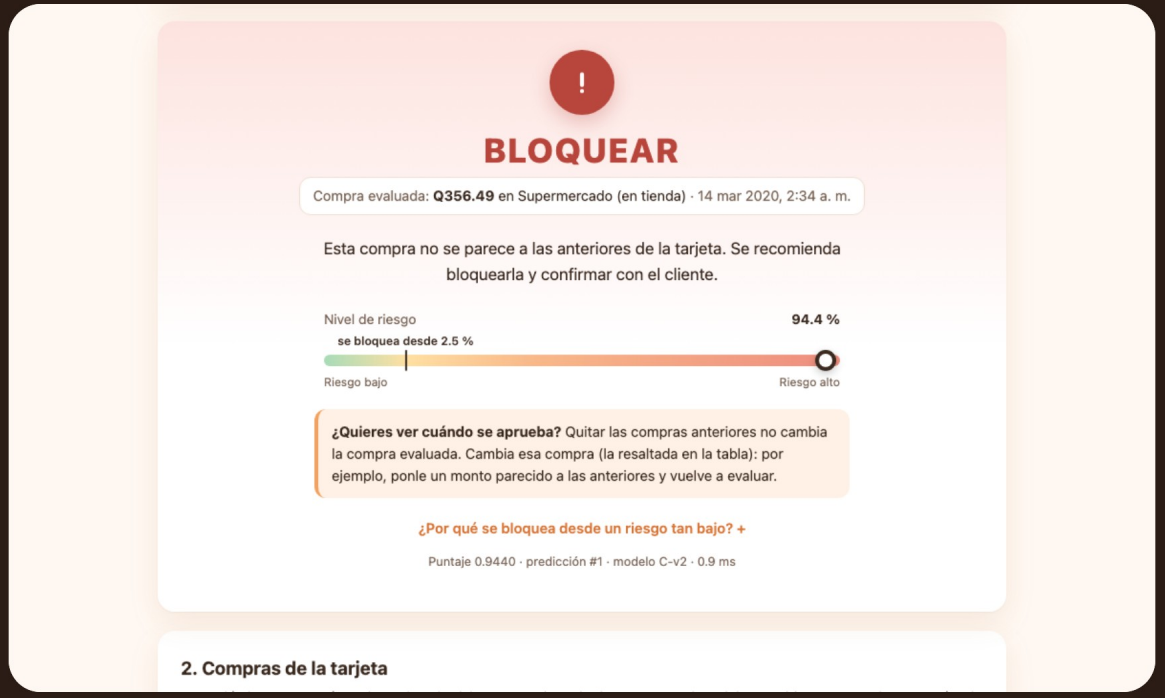
Monitoreo

Cada predicción va a SQLite y se compara con el entrenamiento.

Con qué lo hicimos: PyTorch · FastAPI · SQLite · uv · Docker · Render · GitHub Actions

Veámoslo funcionando

1. **Cliente con compras normales**
   Se aprueba, riesgo casi 0 %
2. **Compra sospechosa**
   Se bloquea: Q356 a las 2:34 a. m.
3. **Le bajamos el monto**
   Vuelve a aprobarse
4. **Abrimos el monitoreo**
   Ahí aparecen las predicciones



Qué demuestra que funciona

32 pruebas

automáticas que corren en GitHub Actions con cada cambio: la API, entradas inválidas, el monitoreo y la página.

1.29 ms

de latencia p95 dentro del servicio publicado. El límite que nos pusimos era 300 ms.

5 escenarios

que probamos los tres en la página publicada, cada uno desde un rol distinto. Quedaron 12 predicciones en el monitoreo.

Q900

en viajes, después de un historial normal, se aprobó. El modelo no bloquea solo porque el monto sea alto.

Nuestra recomendación: dejarlo como piloto

Cuesta 86 % menos que la línea base y responde en milisegundos. Pero se entrenó con datos simulados, así que antes de confiar del todo hay que probarlo con datos del banco.

Lo que todavía no sabemos

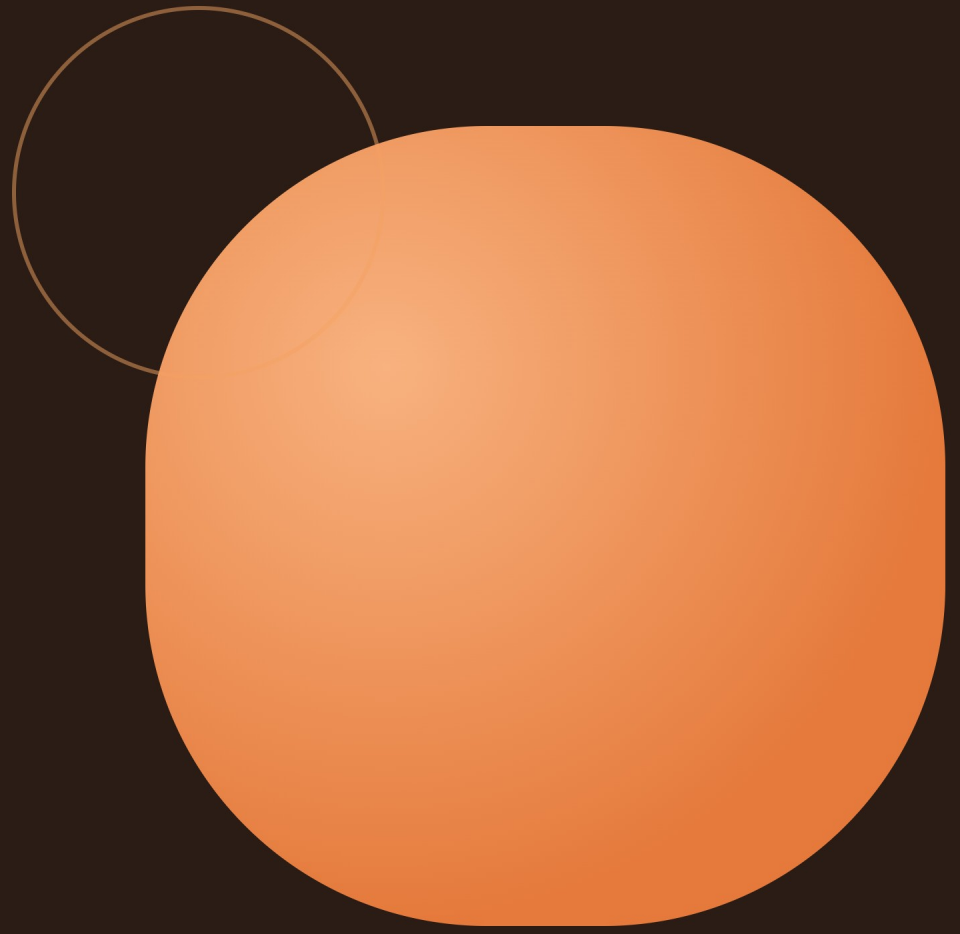
- Cómo le va con fraude real
- Qué pasa con tipos de fraude nuevos
- Si detecta bien en categorías con pocos casos

Lo siguiente

- Reentrenar y validar con datos del banco
- Alertas cuando el monitoreo muestre cambios
- Una base de datos que no se borre

¡Gracias!

¿Preguntas?



fraude-tarjeta.onrender.com

Iris Ayala, Anggie Quezada y Erick Guerra

Deep Learning y Sistemas Inteligentes, UVG 2026